

个人任务及感想（罗展彬）

事件引擎开发者（T3） · 拓扑排序应用软件 · 高级算法原理实践

一、基本信息

- 姓名：罗展彬
- 学号：2024611093
- 组内角色：events 开发者——T3 事件引擎（对应 issue #4）
- Git 身份：xiaoli <xiaolimc@foxmail.com>（GitHub 账号 xiaoliei），下文所有产出均可按此身份在 git 历史中核对

二、本人承担的任务

- issue #4（T3 事件引擎，主责）：app/events/ 模块全部——六类 StepEvent 事件协议、Kahn 步进内核、伪并发时间线（tick 交错 / 暂停 / 单步 / 调速 / 泳道上限），以及配套的行为测试与 golden 事件流证据。
- issue #13（标准图拓扑序数量修正）：对「6 条还是 7 条」的质疑做穷举复核，确认并落地基准修正（详见第四节第 4 条）。
- issue #9（文档批次，按 T8 分工）：04-详细设计第 3 章「事件引擎」撰稿；05-测试报告主责——其中事件引擎用例表、golden 事件流验收一节、缺陷分析的核心素材直接来自我的 T3 交付，文档终稿在交付周由组长统一整理提交入库；另为 03-概要设计提供事件层架构初稿（03 主责为 models 开发者，归属冲突已在 T3 定案 §7/§11 处置留档）。

三、具体产出

代码与证据全部经 PR #17（分支 feat/t3-events，2026-09-20 合入 master，merge commit a40981e）交付：16 条非 merge 提交（4 feat / 1 fix / 5 test / 3 evidence / 3 docs），集中在 2026-09-19 完成。app/events/ 目录共 6 条提交，其中 5 条为本人（另 1 条为 T1 预埋骨架）。

代码（app/events/）

文件	行数	内容
protocol.py	62	6 类 frozen dataclass 事件（Enqueue/Consume/Fork/Complete/DeadEnd/CycleFound）+ event_to_dict() 唯一序列化口径，golden 生成与比对共用
player.py	229	_Branch（每分支一份 Kahn 状态快照）、_Kernel（按分支步进 / 后台晋升 / 环卡死收口）、TopoPlayer（iter_events() 确定性全量流、set_max_completes() 运行期调整）
timeline.py	95	Timeline：拍首快照、拍末补位晋升、泳道上限、tick/pause/resume/step_once/set_speed，3×5 状态机防御式边界

测试与证据

- [app/tests/test_events.py](#) (312 行, 25 个用例、6 个测试类): 确定性互验、调度语义、golden 逐项比对、状态机全表覆盖 (按 T3 定案 §10, 用例追加进 T1 预埋的测试文件, 本人不涉及 CI 配置)。
- [tools/gen_golden_events.py](#) (46 行): golden 事件流生成器, 刻意不重复实现解析。
- golden 三件套 ×3 ([evidence/test-data/](#)): 001-标准五节点图 (61 事件: 24 Enqueue / 24 Consume / 6 Fork / 7 Complete)、002-二节点环 (DeadEnd + CycleFound)、003-环带下游 (`stuck = {A,B,C}`)。

文档与留档

- [evidence/decisions/2026-09-19-T3事件引擎设计定案.md](#): 11 节完整定案, 经 v1→v4 四轮修订。
- [evidence/decisions/2026-09-19-标准图数量修正.md](#): 6→7 修正的复核过程与连带清单 (更新至 v3)。
- [deliverables/04-详细设计.md](#) 第 3 章 (3.1–3.8 八小节, 定稿); [deliverables/03-概要设计.md](#) 事件层初稿供稿。

四、遇到的困难与解决过程

1. 上游未就位时如何开工。T3 启动时 T2 (models) 尚未合入, 而事件内核必须吃图数据。解决: 把依赖面压到最小——`app.events` 只 `from app.models import Graph`、只读 `.nodes/.edges` 两个字段, Kahn 步进完全自研、不调 T2 任何算法 API (同时满足作业「算法自研」红线); 测试手工构造 `Graph(frozenset(...), frozenset(...))`, 自测即 18 绿 + 7 红 (7 红是 T1 预埋、等 T2 合入自然转绿的 `parse` 用例), 全程不被上游进度卡住。
2. 「同输入两次播放逐项相等」在三平台不可保证。Python 集合迭代顺序受 `PYTHONHASHSEED` 影响, 跨进程不固定, 不处理的话跨平台重跑就会偶发不一致。解决: 定总则——一切集合遍历与事件发射顺序一律 `sorted()` 字典序; 另写互验测试锁死 `iter_events()` 全量流 $\equiv$ 无限泳道 Timeline 逐 tick 拼接, 两种播放口径数学上同源, 渲染层拿到的事件永远自洽。
3. 宽并行图的分支爆炸 (最险的一个)。`max_completes` 按 Complete 计数点火在宽并行图上太晚: 首条 Complete 之前 Fork 已按组合数爆炸, 实测 30 节点双链图跑 26 秒零产出 (该图全序数约 1.5 亿条)。解决 (定案 v4): 前置保险丝——Fork 前检查「已完成数 + 活跃分支数 $\geq$ cap 即不再开新分支」(每条活跃分支至多 1 个 Complete, 上界自洽); 500 例随机图回归验证, 并给保险丝测试加了「活跃分支峰值 $\leq$ cap」断言, 失效可检出。
4. 自己把标准答案数错了。标准五节点图的拓扑序我最初按 6 条写基准, Asukadaisiki 在 issue #13 提出 7 条。穷举 $5!=120$ 排列复核后确认我漏了 `A E B C D`。结论定为「算法不得故意漏解, 修正基准而不是让算法迁就错误计数」, 连带修正常量、用例与文档 (修正记录与连带清单留档至 v3)。这次纠错让我对「先证明、再写码」有了切身体会。
5. Fork 语义的歧义。某步候选有 $k \geq 2$ 个节点时, 原分支消费字典序最小者、其余 $k-1$ 个备选各开一条新分支; 但「新分支的首步」是重新按字典序自选、还是必须消费 Fork 时绑定的备选节点, 契约没有写死。模拟后发现前一种解读会让所有分支收敛成同一条序 (Complete 恒为 1), 只有「绑定备选」能得到标准图的 7 次 Complete——据此把该语义定进定案 §2。类似的细节还有: `CycleFound` 必须拼在使全部分支终止的最后一拍事件列表末尾, 否则消费方 `while state != "finished": tick()` 会漏收这条事件。

五、个人感想

这个项目给我最大的收获是**确定性思维**。以前写程序的标准是「能跑出正确结果」, 做事件引擎才明白, 教学演示软件的标准是「任意平台、任意时刻、重跑一万次, 动画逐帧一致」——为此每一步候选选择都要定死字典序, 每个集合遍历都要排序, 连「后台分支晋升补发 Enqueue」这种细节都要论证它为什么不构成重复事件。把标准从「对」提到「可复现」, 是这次实践教我的第一课。

第二是**契约先行和留档的价值**。T3 是全组最早开工的任务之一, 开工时上游没合入、下游在等我, 靠 T1 定下的接口契约和预埋红测试, 五个人从契约定稿 (09–15) 到各自模块交付 (09–19/09–20) 基本互不阻塞地并行推进。我自己的设计定案改了

四版，每一版修订都是交叉审查揪出的真问题——尤其 v4 前置保险丝，是终审才发现的 §8×§9 组合盲区。如果没有「决策必须留档」的组规，这些推理过程一周后就会蒸发，只剩一句「当时就是这么定的」。

第三是**诚实面对自己的错误**。把基准拓扑序数成 6 条这件事提醒我：人对组合对象的直觉很不可靠，穷举验证不丢人，数错了还嘴硬才丢人。修正基准而不是让算法迁就错误计数，这个决定后来被写进了决策档案，我觉得它比任何一段代码都更能代表这次实践教给我的工程态度。

最后，把「播放动画」抽象成「确定性事件流」的分层设计让我真正体会了架构的意义：渲染层只认事件、不懂算法；我写的事件层不碰 Qt、也不依赖算法库的现成排序。每一层都可以被单独测试、单独替换——这份可单独验证的踏实感，是我对「软件工程」最具体的一次理解。

附：核对指引

```
# 本人全部提交 (16 条, 无 merge)
git log --author="xiaolimc@foxmail.com" --oneline --no-merges

# 关键提交索引
#   3d02b0d 事件协议 event_to_dict          c9cdbdd Kahn 步进内核
#   5a1be3a 伪并发时间线                    e19d82e golden 三件套 + 生成器
#   d678ead T3 行为测试 (25 用例)           c3adcb8 标准图序数 6→7 修正
#   59461fd 分支数前置保险丝                e98cc65 03 初稿 / 04 事件章供稿
# PR: #17 feat/t3-events (merge commit a40981e)
```